

LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

with LangGraph & Human-in-the-Loop (HITL)

1. Module-Level Design

The system is decomposed into 8 cohesive modules, each with a single responsibility. Below is the detailed design of each module.

1.1 Document Processing Module

Attribute	Details
Class	DocumentProcessor
Responsibility	Load and extract text from PDF files
Input	PDF file path(s) — string or list of strings
Output	List of LangChain Document objects with page_content and metadata
Dependencies	langchain_community.document_loaders.PyPDFLoader
Error Handling	FileNotFoundError, PDFParseError — logged and skipped

Key Method Signature:

```
def load_documents(self, pdf_paths: List[str]) -> List[Document]:  
    # Iterates paths, loads each PDF, appends metadata  
    # Returns: List[Document(page_content=str, metadata={source, page})]
```

1.2 Chunking Module

Attribute	Details
Class	ChunkingEngine
Responsibility	Split documents into semantically meaningful chunks
Input	List[Document] from DocumentProcessor
Output	List[Document] — each representing one chunk with enriched metadata
Config	chunk_size=800, chunk_overlap=100, separators=[paragraph, sentence, word]
Dependencies	langchain.text_splitter.RecursiveCharacterTextSplitter

Key Method Signature:

```
def chunk_documents(self, docs: List[Document]) -> List[Document]:  
    # Applies recursive splitting, adds chunk_index to metadata  
    # Returns: List[Document] with chunk-level metadata
```

1.3 Embedding Module

Attribute	Details
Class	EmbeddingEngine
Responsibility	Convert text chunks to dense vector representations
Model	sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)
Input	List[str] — chunk text content
Output	List[List[float]] — embedding vectors
Batch Size	32 chunks per batch to manage memory
Fallback	OpenAI text-embedding-3-small (if local model unavailable)

1.4 Vector Storage Module

Attribute	Details
Class	VectorStoreManager
Responsibility	Persist and manage embedding storage in ChromaDB
Store	ChromaDB (persistent, local disk at ./chroma_db/)
Collection Name	customer_support_kb
Operations	upsert_chunks(), delete_collection(), get_collection_stats()
Deduplication	SHA-256 hash of chunk content used as document ID

1.5 Retrieval Module

Attribute	Details
Class	RetrieverEngine
Responsibility	Query ChromaDB and return ranked relevant chunks
Strategy	Cosine similarity search with score threshold filtering
Default k	5 documents
Score Threshold	0.35 (below = irrelevant, triggers escalation consideration)
Output	List[Tuple[Document, float]] — (chunk, similarity_score)

1.6 Query Processing Module

Attribute	Details
Class	QueryProcessor

Responsibility	Format prompt, call LLM, parse structured output
Input	query: str, context_chunks: List[Document]
Output	QueryResult(answer: str, confidence: float, sources: List[str])
Prompt Template	System role + Context block + User query + Output format instructions
Confidence Calc	Average of retrieval scores, modulated by LLM self-reported confidence

1.7 Graph Execution Module

Attribute	Details
Class	RAGWorkflowGraph
Responsibility	Compile and execute the LangGraph state machine
Nodes	input_node, retrieve_node, generate_node, route_node, output_node, hitl_node
Edges	input → retrieve → generate → route → (output hitl)
State Object	RAGState TypedDict (see Data Structures)
Interrupt Point	At hitl_node — graph pauses awaiting human input

1.8 HITL Module

Attribute	Details
Class	HITLHandler
Responsibility	Manage escalation lifecycle and human agent interaction
Trigger	Confidence < 0.4 OR no chunks retrieved OR query classified as complex
Queue Type	In-memory queue (production: Redis or RabbitMQ)
Agent Interface	CLI prompt (production: Slack bot or web dashboard)
Resume	Human provides answer string; graph resumes from hitl_node

2. Data Structures

2.1 Document Representation

```
class Document:
    page_content: str          # raw text of the chunk
    metadata: Dict = {
        "source": str,        # file path
        "page": int,          # page number in PDF
        "chunk_index": int,    # position within document
        "doc_id": str,         # SHA-256 hash for deduplication
        "char_count": int,     # number of characters in chunk
    }
```

2.2 Chunk Format

```
ChunkMetadata = TypedDict('ChunkMetadata', {
    "source": str,
    "page": int,
    "chunk_index": int,
    "doc_id": str,          # SHA-256 of content
    "embedding_model": str, # model used for embedding
    "created_at": str,      # ISO timestamp
})
```

2.3 Embedding Structure

```
EmbeddingRecord = {
    "id": str,                # doc_id
    "embedding": List[float], # 384-dim vector (MiniLM) or 1536-dim (OpenAI)
    "document": str,          # chunk text
    "metadata": ChunkMetadata
}
```

2.4 Query-Response Schema

```
class QueryResult(BaseModel):
    query: str
    answer: str
    confidence: float          # 0.0 - 1.0
    sources: List[str]         # list of source file names
    escalated: bool            # True if routed to HITL
    retrieval_scores: List[float] # similarity scores of retrieved chunks
    latency_ms: int            # total processing time
```

2.5 State Object for Graph

```
class RAGState(TypedDict):
    query: str                # original user query
```

```
processed_query: str          # cleaned, normalized query
retrieved_chunks: List[Document]  # top-k results
retrieval_scores: List[float]    # similarity scores
llm_response: str              # raw LLM output
confidence: float               # computed confidence score
final_answer: str              # answer delivered to user
escalated: bool                 # escalation flag
human_answer: Optional[str]     # answer from HITL
error: Optional[str]            # error message if any
```

3. Workflow Design (LangGraph)

3.1 Node Definitions

Node Name	Responsibility
input_node	Validate query, normalize whitespace, detect language, update state.processed_query
retrieve_node	Embed processed_query, query ChromaDB, update state.retrieved_chunks and state.retrieval_scores
generate_node	Build prompt from chunks, call LLM, compute confidence, update state.llm_response
route_node	Conditional: return 'output' if confidence >= 0.4 AND chunks found, else return 'hitl'
output_node	Format final answer, add source citations, update state.final_answer
hitl_node	Interrupt graph, queue escalation, await human_answer, update state.final_answer

3.2 Edge Flow

```
graph = StateGraph(RAGState)
graph.add_node('input', input_node)
graph.add_node('retrieve', retrieve_node)
graph.add_node('generate', generate_node)
graph.add_node('route', route_node)
graph.add_node('output', output_node)
graph.add_node('hitl', hitl_node)

graph.set_entry_point('input')
graph.add_edge('input', 'retrieve')
graph.add_edge('retrieve', 'generate')
graph.add_edge('generate', 'route')
graph.add_conditional_edges('route', routing_function,
    {'output': 'output', 'hitl': 'hitl'})
graph.add_edge('output', END)
graph.add_edge('hitl', END)

compiled_graph = graph.compile(interrupt_before=['hitl'])
```

4. Conditional Routing Logic

4.1 Routing Decision Function

```
def routing_function(state: RAGState) -> str:
    confidence = state['confidence']
    has_chunks = len(state['retrieved_chunks']) > 0
    avg_score = sum(state['retrieval_scores']) / max(len(state['retrieval_scores']),
1)

    if confidence >= 0.4 and has_chunks and avg_score >= 0.35:
        return 'output'    # Auto-answer
    else:
        return 'hitl'      # Escalate to human
```

4.2 Escalation Criteria

Condition	Action
confidence < 0.4	Escalate — LLM uncertainty too high
retrieved_chunks == []	Escalate — no relevant knowledge found
avg_score < 0.35	Escalate — retrieved content not sufficiently relevant
Query contains complaint/refund keywords	Escalate — sensitive topic requires human
Query length > 500 chars (complex)	Escalate — multi-part question needs expert
confidence >= 0.4 AND avg_score >= 0.35	Auto-answer — proceed to output_node

5. HITL Design

5.1 When Escalation is Triggered

The HITL mechanism activates under the following conditions (evaluated in `route_node`):

- Retrieval confidence score falls below 0.40 threshold
- No relevant chunks were retrieved from the vector store
- Query is classified as sensitive (billing disputes, legal queries, complaints)
- Query is identified as multi-intent or highly ambiguous

5.2 What Happens After Escalation

```
# Step 1: Graph pauses at hitl_node (LangGraph interrupt)
# Step 2: Escalation payload is created:
EscalationPayload = {
    "query": state["query"],
    "partial_answer": state["llm_response"],
    "retrieved_context": [c.page_content for c in state["retrieved_chunks"]],
    "confidence": state["confidence"],
    "timestamp": datetime.utcnow().isoformat(),
    "escalation_id": uuid4()
}
# Step 3: Payload sent to human agent queue
# Step 4: System notifies agent (CLI prompt / webhook)
# Step 5: Agent reviews and provides verified_answer
```

5.3 How Human Response is Integrated

```
# Human agent submits answer:
human_answer = input('Agent Response: ')

# Graph is resumed with human answer injected into state:
compiled_graph.update_state(
    config=thread_config,
    values={'human_answer': human_answer, 'escalated': True}
)
compiled_graph.invoke(None, config=thread_config)
# Graph resumes from hitl_node, routes to output_node
```

6. API / Interface Design

6.1 Input Format

```
POST /api/query
Content-Type: application/json
{
  "query": "What is your refund policy?",
  "session_id": "user-abc-123",          // optional, for conversation threading
  "metadata": { "user_type": "premium" } // optional
}
```

6.2 Output Format

```
HTTP 200 OK
{
  "query": "What is your refund policy?",
  "answer": "Our refund policy allows returns within 30 days...",
  "confidence": 0.82,
  "sources": ["company_policy.pdf (page 3)"],
  "escalated": false,
  "latency_ms": 1240
}
```

```
// Escalated response:
{
  "answer": "A human agent will respond within 2 business hours.",
  "escalated": true,
  "escalation_id": "esc-9f4a-...",
  "confidence": 0.21
}
```

6.3 CLI Interface Flow

```
$ python main.py
> Ingesting knowledge base...
> Knowledge base ready. 247 chunks indexed.
> Customer Support Assistant Ready. Type 'quit' to exit.
> You: What is your return policy?
> [Retrieving...] [Generating...] [Confidence: 0.87]
> Assistant: Based on our policy document, returns are accepted within...
>           Source: policy_manual.pdf (Page 4)
```

7. Error Handling

Error Scenario	Handling Strategy
PDF file not found	Log FileNotFoundError, skip file, continue with other PDFs
PDF parse failure	Catch PDFParseError, log warning, skip corrupt pages
Embedding model unavailable	Retry 3x with exponential backoff; fall back to OpenAI embeddings
ChromaDB connection failure	Raise RuntimeError with descriptive message; block startup
No chunks retrieved	Set confidence=0.0, trigger HITL escalation immediately
LLM API timeout	Retry 2x; if still failing, escalate to HITL with timeout note
LLM API rate limit	Implement token bucket rate limiter; queue requests
Graph state corruption	Validate state schema at each node entry; raise GraphStateError
HITL timeout (no agent)	Auto-reply: 'Agent unavailable, ticket created #ID'; log escalation